



Frontend

Guía completa del frontend

Construimos el frontend de Dolt App paso a paso. Usamos Tailwind CSS v4 con un tema dark/orange custom, react-hook-form para todos los formularios, y la URL del backend se lee desde una variable de entorno de Vite para que local y producción funcionen sin tocar el código.

Objetivo

- ver una portada
- registrarse e iniciar sesión con validación en cliente
- guardar sesión y proteger rutas privadas
- dashboard con CRUD de tareas, filtros y ordenación
- perfil con edición de nombre, contraseña y avatar

Orden de construcción

1. crear el proyecto e instalar dependencias
2. configurar Tailwind v4 + tema custom
3. crear `api.js` con variable de entorno
4. crear el contexto de autenticación
5. crear componentes base
6. crear páginas simples
7. crear páginas conectadas al backend
8. montar rutas en `main.jsx`

1. Crear el proyecto e instalar dependencias

```
npm create vite@latest doitapp -- --template react
cd doitapp
npm install
```

```
npm install react-router-dom react-hook-form
npm install tailwindcss @tailwindcss/vite
```

Por qué estas librerías

- `react-router-dom` → navegación entre páginas
- `react-hook-form` → formularios sin `useState` por campo, con validación declarativa
- `tailwindcss` + `@tailwindcss/vite` → estilos utility-first integrados en el pipeline de Vite

2. Estructura del proyecto

```
frontend/
├── src/
│   ├── components/
│   │   ├── Navbar.jsx
│   │   ├── ProtectedRoute.jsx
│   │   ├── TaskForm.jsx
│   │   ├── TaskFilters.jsx
│   │   ├── TaskList.jsx
│   │   ├── TaskCard.jsx
│   │   ├── UsernameForm.jsx
│   │   ├── PasswordForm.jsx
│   │   └── AvatarForm.jsx
│   ├── context/
│   │   └── AuthContext.jsx
│   ├── pages/
│   │   ├── Home.jsx
│   │   ├── Login.jsx
│   │   ├── Register.jsx
│   │   ├── Dashboard.jsx
│   │   ├── Profile.jsx
│   │   └── NotFound.jsx
│   ├── services/
│   │   └── api.js
│   └── test/
│       ├── setup.js
│       ├── TaskCard.test.jsx
│       └── TaskList.test.jsx
```

```

| | | TaskForm.test.jsx
| | | └─ TaskFilters.test.jsx
| | └─ App.jsx
| | └─ main.jsx
| | └─ index.css
| └─ .env
| └─ .gitignore
| └─ package.json
└─ vite.config.js

```

Ya no hay `Navbar.css` — los estilos van directamente con clases de Tailwind.

3. Configurar Tailwind v4 y el tema

`vite.config.js`

```

import { defineConfig } from "vite";
import react          from "@vitejs/plugin-react";
import tailwindcss    from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
  test: {
    environment: "jsdom",
    setupFiles:  "./src/test/setup.js",
    globals:    true,
  },
});

```

`src/index.css`

```
@import "tailwindcss";
```

```
/* — Tema custom de DoIt App —————
```

En Tailwind v4 los tokens del tema se definen con variables CSS en `@theme`.

Quedan disponibles como clases: `bg-brand`, `text-brand`, `bg-surface`, etc.

```

    */
@theme {
  --color-brand:          #f97316;    /* naranja – acento principal */
  --color-brand-dark:     #ea580c;    /* naranja oscuro – hover */
  --color-bg:             #09090b;    /* zinc-950 – fondo de la app */
  --color-surface:        #18181b;    /* zinc-900 – navbar y tarjetas */
  --color-input:          #27272a;    /* zinc-800 – fondo de inputs */
  --color-border:         #3f3f46;    /* zinc-700 – bordes */
  --color-muted:          #a1a1aa;    /* zinc-400 – texto secundario */
  --color-error:          #f87171;    /* red-400 – errores */
  --color-success:        #4ade80;    /* green-400 – éxito */
}

```

Con esto tenemos clases como `bg-brand`, `text-brand`, `hover:bg-brand-dark`, `bg-bg`, `bg-surface`, `bg-input`, `border-border`, `text-muted`, `text-error`, `text-success`.

`.gitignore` del frontend

```

node_modules
dist
.env
.env.production

```

`.env` (local)

```
VITE_API_URL=http://localhost:3000/api
```

Nota: el backend corre en el puerto **3000**. El frontend lo lee desde `import.meta.env.VITE_API_URL`. Las variables `VITE_*` se incrustan en el bundle en tiempo de build, así que si la cambias en Vercel necesitas hacer un nuevo deploy.

4. `src/services/api.js`

```
// leemos la URL del backend desde la variable de entorno de Vite
// en local → http://localhost:3000/api (del archivo .env)
// en producción → la URL de Render (configurada en Vercel)
const API_URL = import.meta.env.VITE_API_URL;

// ——— USERS ———

const registerUser = async (formData) => {
  const res = await fetch(`${API_URL}/users/register`, {
    method: "POST",
    body: formData, // FormData → el navegador gestiona el Content-Type multipart
  });
  return res.json();
};

const loginUser = async (data) => {
  const res = await fetch(`${API_URL}/users/login`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(data),
  });
  return res.json();
};

const getMe = async (token) => {
  const res = await fetch(`${API_URL}/users/me`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  return res.json();
};

const updateUsername = async (token, username) => {
  const res = await fetch(`${API_URL}/users/username`, {
```

```

        method: "PUT",
        headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
        body: JSON.stringify({ username }),
    });
    return res.json();
};

const updatePassword = async (token, currentPassword, newPassword) => {
    const res = await fetch(`${API_URL}/users/password`, {
        method: "PUT",
        headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
        body: JSON.stringify({ currentPassword, newPassword }),
    });
    return res.json();
};

const updateAvatar = async (token, formData) => {
    const res = await fetch(`${API_URL}/users/avatar`, {
        method: "PUT",
        headers: { Authorization: `Bearer ${token}` }, // sin Content-Type: el navegador lo gestiona
        body: formData,
    });
    return res.json();
};

// — TASKS —

const getTasks = async (token, filters = {}) => {
    const params = new URLSearchParams();
    if (filters.status) params.append("status", filters.status);
    if (filters.date) params.append("date", filters.date);

```

```

e);
  if (filters.sort) params.append("sort", filters.sort);

  const res = await fetch(`${API_URL}/tasks?${params.toString()}`, {
    headers: { Authorization: `Bearer ${token}` },
  });
  return res.json();
};

const createTask = async (token, taskData) => {
  const res = await fetch(`${API_URL}/tasks`, {
    method: "POST",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
    body: JSON.stringify(taskData),
  });
  return res.json();
};

const updateTask = async (token, id, taskData) => {
  const res = await fetch(`${API_URL}/tasks/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json", Authorization: `Bearer ${token}` },
    body: JSON.stringify(taskData),
  });
  return res.json();
};

const deleteTask = async (token, id) => {
  const res = await fetch(`${API_URL}/tasks/${id}`, {
    method: "DELETE",
    headers: { Authorization: `Bearer ${token}` },
  });
  return res.json();
};

```

```
export {
  registerUser, loginUser, getMe,
  updateUsername, updatePassword, updateAvatar,
  getTasks, createTask, updateTask, deleteTask,
};
```

5. `src/context/AuthContext.jsx`

```
import { createContext, useCallback, useContext, useEffect,
useState } from "react";
import { getMe } from "../services/api";

const AuthContext = createContext();

export const AuthProvider = ({ children }) => {
  const [token, setToken] = useState(localStorage.getItem(
    "token" ) || "");
  const [user, setUser] = useState(null);

  // useCallback: loadUser se expone en el contexto y lo us
  an UsernameForm y AvatarForm
  // sin useCallback cambiaría de referencia en cada render
  y rompería los efectos
  const loadUser = useCallback(async () => {
    if (!token) { setUser(null); return; }

    const res = await getMe(token);

    // si la respuesta no tiene _id, el token no es válido
    o ha expirado
    // getMe devuelve el usuario (con _id) si va bien, o {
    message } si falla
    // comprobar _id es más robusto que comprobar una clave
    de error que puede variar
    if (!res._id) {
      localStorage.removeItem("token");
    }
  }, [token]);
```



```

        setToken("");
        setUser(null);
        return;
    }

    setUser(res);
}, [token]);

useEffect(() => { loadUser(); }, [loadUser]);

const login = useCallback((newToken, userData) => {
    localStorage.setItem("token", newToken);
    setToken(newToken);
    setUser(userData);
}, []);

const logout = useCallback(() => {
    localStorage.removeItem("token");
    setToken("");
    setUser(null);
}, []);

return (
    <AuthContext.Provider value={{ token, user, login, logout, loadUser }}>
        {children}
    </AuthContext.Provider>
);
};

export const useAuth = () => useContext(AuthContext);

```

6. Componentes base

6.1 Navbar

Sin `Navbar.css` — los estilos van con clases de Tailwind. Para el enlace activo usamos la función `className` de `NavLink`.

```

import { memo } from "react";
import { NavLink } from "react-router-dom";

// clases compartidas para todos los enlaces
const linkClass = "uppercase tracking-widest text-sm transition-colors";

// función que devuelve las clases según si el enlace está activo o no
const navLinkClass = ({ isActive }) =>
  `${linkClass} ${isActive ? "text-brand" : "text-muted hover:text-brand"}`;

const Navbar = memo(({ token, onLogout }) => {
  return (
    <nav className="bg-surface border-b border-border px-6 py-4">
      <ul className="flex gap-6 items-center list-none">
        <li>
          <NavLink to="/" end className={navLinkClass}>Inicio</NavLink>
        </li>

        {token ? (
          <>
            <li><NavLink to="/dashboard" className={navLinkClass}>Dashboard</NavLink></li>
            <li><NavLink to="/profile" className={navLinkClass}>Perfil</NavLink></li>
            <li>
              <button
                onClick={onLogout}
                className="px-3 py-1.5 rounded-lg border border-dashed border-border text-muted hover:bg-surface hover:text-white uppercase tracking-widest text-sm transition-colors"
              >
                Salir
              </button>
            </li>
          </>
        ) : null}
      </ul>
    </nav>
  );
});

```

```

        </button>
      </li>
    </>
  ) : (
    <>
      <li><NavLink to="/login" className={navLinkC
lass}>Login</NavLink></li>
      <li><NavLink to="/register" className={navLinkC
lass}>Registro</NavLink></li>
    </>
  )}
</ul>
</nav>
);
});

Navbar.displayName = "Navbar";
export default Navbar;

```

Por qué `end` en el enlace a `/`

Sin `end`, el enlace a `/` aparecería siempre activo porque `/dashboard` también empieza por `/`. Con `end` solo se activa cuando la ruta es exactamente `/`.

6.2 ProtectedRoute

```

import { Navigate, Outlet } from "react-router-dom";
import { useAuth } from "../context/AuthContext";

const ProtectedRoute = () => {
  const { token } = useAuth();
  // replace evita que la ruta protegida quede en el histor
  ial del navegador
  if (!token) return <Navigate to="/login" replace />;
  return <Outlet />;
};

export default ProtectedRoute;

```

6.3 TaskFilters

No usa react-hook-form (los selects los controla el Dashboard, no el propio componente).

```
import { memo } from "react";

// clases reutilizables para los controles de filtro
const selectClass =
  "w-full px-3 py-2 rounded-lg bg-input border border-borde
r text-white focus:outline-none focus:border-brand transiti
on-colors";

const TaskFilters = memo(({ filters, setFilters }) => {
  return (
    <div className="flex flex-wrap gap-3 justify-center">

      <select
        value={filters.status}
        onChange={(ev) => setFilters({ ...filters, status:
ev.target.value })}
        className={selectClass}
      >
        <option value="">Todos los estados</option>
        <option value="pending">Pendientes</option>
        <option value="completed">Completadas</option>
      </select>

      <input
        type="date"
        value={filters.date}
        onChange={(ev) => setFilters({ ...filters, date: e
v.target.value })}
        className={selectClass}
      />

      <select
        value={filters.sort}
        onChange={(ev) => setFilters({ ...filters, sort: e
```

```

v.target.value }))}
    className={selectClass}
  >
    <option value="">Sin ordenar</option>
    <option value="date_asc">Fecha ↑</option>
    <option value="date_desc">Fecha ↓</option>
    <option value="status_asc">Estado ↑</option>
    <option value="status_desc">Estado ↓</option>
  </select>
</div>
);
});

TaskFilters.displayName = "TaskFilters";
export default TaskFilters;

```

6.4 TaskForm — con react-hook-form

```

import { memo, useEffect } from "react";
import { useForm } from "react-hook-form";

// clases reutilizables
const inputClass =
  "w-full px-3 py-2 rounded-lg bg-input border border-borde
r text-white placeholder:text-muted focus:outline-none focu
s:border-brand transition-colors";
const errorClass = "text-error text-xs mt-1";
const btnPrimary =
  "px-4 py-2 rounded-lg bg-brand hover:bg-brand-dark text-w
hite uppercase tracking-widest text-sm font-medium transiti
on-colors";
const btnOutline =
  "px-4 py-2 rounded-lg border border-dashed border-border
text-muted hover:bg-surface uppercase tracking-widest text-
sm transition-colors";

const TaskForm = memo(({ onSubmitTask, taskToEdit, clearTas
kToEdit }) => {

```

```

const {
  register,
  handleSubmit,
  formState: { errors },
  reset,
  setValue,
} = useForm({ defaultValues: { title: "", date: "", status: "pending" } });

// cuando taskToEdit cambia, rellenos el formulario con
sus datos
useEffect(() => {
  if (taskToEdit) {
    setValue("title", taskToEdit.title);
    setValue("date", taskToEdit.date?.slice(0, 10)); //
    "2026-04-14T..." → "2026-04-14"
    setValue("status", taskToEdit.status);
  }
}, [taskToEdit, setValue]);

const onSubmit = (data) => {
  onSubmitTask(data);
  reset();
};

const handleCancel = () => {
  clearTaskToEdit();
  reset();
};

return (
  <form onSubmit={handleSubmit(onSubmit)} className="flex
flex-col gap-3 w-full max-w-sm">
    <div>
      <input
        type="text"
        placeholder="Título de la tarea"
        className={inputClass}

```

```

        {...register("title", {
            required: "El título es obligatorio",
            minLength: { value: 3, message: "Mínimo 3 caracteres" },
        })}
    />
    {errors.title && <p className={errorClass}>{errors.title.message}</p>}
</div>

<div>
    <input
        type="date"
        className={inputClass}
        {...register("date", { required: "La fecha es obligatoria" })}
    />
    {errors.date && <p className={errorClass}>{errors.date.message}</p>}
</div>

<select className={inputClass} {...register("status")}>
    <option value="pending">Pendiente</option>
    <option value="completed">Completada</option>
</select>

<button type="submit" className={btnPrimary}>
    {taskToEdit ? "Guardar cambios" : "Crear tarea"}
</button>

{taskToEdit && (
    <button type="button" onClick={handleCancel} className={btnOutline}>
        Cancelar edición
    </button>
)}
</form>

```

```

    );
  });

  TaskForm.displayName = "TaskForm";
  export default TaskForm;

```

6.5 TaskCard

```

import { memo, useCallback } from "react";

const TaskCard = memo(({ task, onEdit, onDelete }) => {
  const handleEdit = useCallback(() => onEdit(task),
    [task, onEdit]);
  const handleDelete = useCallback(() => onDelete(task._id),
    [task._id, onDelete]);

  const isCompleted = task.status === "completed";

  return (
    <article className="flex flex-col gap-3 p-5 rounded-xl
    bg-surface border border-border">
      <h3 className={`text-white font-semibold ${isCompleted
    ? "line-through text-muted" : ""}`}>
        {task.title}
      </h3>

      <p className="text-muted text-sm">{task.date?.slice
    (0, 10)}</p>

      <span
        className={`self-start px-2 py-0.5 rounded-full tex
    t-xs font-medium ${
          isCompleted
            ? "bg-green-900/40 text-success"
            : "bg-orange-900/40 text-brand"
        }`>
        >
        {isCompleted ? "Completada" : "Pendiente"}
      </span>
    </article>
  );

```



```

    </span>

    <div className="flex gap-2 mt-1">
      <button
        onClick={handleEdit}
        className="flex-1 px-3 py-1.5 rounded-lg border border-muted text-muted hover:bg-input hover:text-white text-sm transition-colors"
      >
        Editar
      </button>
      <button
        onClick={handleDelete}
        className="flex-1 px-3 py-1.5 rounded-lg border border-red-900 text-red-400 hover:bg-red-900/20 text-sm transition-colors"
      >
        Borrar
      </button>
    </div>
  </article>
);
});

TaskCard.displayName = "TaskCard";
export default TaskCard;

```

6.6 TaskList

```

import { memo } from "react";
import TaskCard from "../TaskCard";

const TaskList = memo(({ tasks, onEdit, onDelete }) => {
  if (!tasks.length) {
    return (
      <p className="text-muted text-center py-10">No hay tareas todavía</p>
    );
  }

```

```

    }

    return (
      <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid
-cols-3 gap-4 w-full">
        {tasks.map((task) => (
          <TaskCard key={task._id} task={task} onEdit={onEdit}
onDelete={onDelete} />
        ))}
      </div>
    );
  });
});

TaskList.displayName = "TaskList";
export default TaskList;

```

7. Páginas simples

7.1 Home

```

import { Link } from "react-router-dom";

const Home = () => (
  <section className="flex flex-col items-center justify-center min-h-[80vh] gap-6 px-4">
    <h1 className="text-5xl font-bold text-brand tracking-tight">DoIt App</h1>
    <p className="text-muted text-lg">Gestiona tus tareas de forma sencilla</p>
    <div className="flex gap-4 mt-4">
      <Link
        to="/login"
        className="px-6 py-2.5 rounded-lg bg-brand hover:bg-brand-dark text-white uppercase tracking-widest text-sm font-medium transition-colors"
      >
        Acceder
      </Link>
    </div>
  </section>
);

```

```

    </Link>
    <Link
      to="/register"
      className="px-6 py-2.5 rounded-lg border border-dashed border-border text-muted hover:bg-surface hover:text-white uppercase tracking-widest text-sm transition-colors"
    >
      Crear cuenta
    </Link>
  </div>
</section>
);

export default Home;

```

7.2 NotFound

```

import { Link } from "react-router-dom";

const NotFound = () => (
  <section className="flex flex-col items-center justify-center min-h-[80vh] gap-4">
    <h1 className="text-8xl font-bold text-brand">404</h1>
    <p className="text-muted text-lg">Página no encontrada</p>
    <Link
      to="/"
      className="mt-4 px-5 py-2 rounded-lg bg-surface border border-border text-muted hover:text-white transition-colors"
    >
      Volver al inicio
    </Link>
  </section>
);

export default NotFound;

```

8. Páginas conectadas al backend

8.1 Register

```
import { useForm } from "react-hook-form";
import { useNavigate } from "react-router-dom";
import { registerUser } from "../services/api";

const inputClass = "w-full px-3 py-2 rounded-lg bg-input border border-border text-white placeholder:text-muted focus:outline-none focus:border-brand transition-colors";
const errorClass = "text-error text-xs mt-1";
const btnPrimary = "w-full px-4 py-2.5 rounded-lg bg-brand hover:bg-brand-dark text-white uppercase tracking-widest text-sm font-medium transition-colors";

const Register = () => {
  const navigate = useNavigate();
  const { register, handleSubmit, formState: { errors }, setError } = useForm();

  const onSubmit = async (data) => {
    const formData = new FormData();
    formData.append("username", data.username);
    formData.append("email", data.email);
    formData.append("password", data.password);
    // data.avatar es un FileList; si no se seleccionó ninguno el backend usa el placeholder
    if (data.avatar?.[0]) formData.append("avatar", data.avatar[0]);

    const res = await registerUser(formData);

    if (res.error) {
      setError("root", { message: res.error });
      return;
    }
  }
}
```

```

        navigate("/login");
    };

    return (
        <section className="flex flex-col items-center justify-
center min-h-[80vh] px-4">
            <div className="w-full max-w-sm bg-surface rounded-2x
l p-8 border border-border">
                <h1 className="text-brand text-2xl font-bold upperc
ase tracking-widest text-center mb-6">
                    Crear cuenta
                </h1>

                <form onSubmit={handleSubmit(onSubmit)} className
="flex flex-col gap-4">
                    <div>
                        <input type="text" placeholder="Username" class
Name={inputClass}
                            {...register("username", {
                                required: "El username es obligatorio",
                                minLength: { value: 3, message: "Mínimo 3 c
aracteres" },
                            })}
                        />
                        {errors.username && <p className={errorClass}>
{errors.username.message}</p>}
                    </div>

                    <div>
                        <input type="email" placeholder="Email" classNa
me={inputClass}
                            {...register("email", {
                                required: "El email es obligatorio",
                                pattern: { value: /^[^\s@]+@[^\s@]+\.[^\s
@]+$/ , message: "Email no válido" },
                            })}
                        />
                        {errors.email && <p className={errorClass}>{err

```

```

    {errors.email.message}</p>
    </div>

    <div>
      <input type="password" placeholder="Contraseña"
      className={inputClass}
      {...register("password", {
        required: "La contraseña es obligatoria",
        minLength: { value: 8, message: "Mínimo 8 c
aracteres" },
      })}
      />
      {errors.password && <p className={errorClass}>
{errors.password.message}</p>}
    </div>

    <div>
      <label className="text-muted text-sm block mb-
1">Avatar (opcional)</label>
      <input type="file" accept="image/*" className=
{\`${inputClass} file:mr-3 file:text-muted file:bg-transpare
nt file:border-0`}
      {...register("avatar")}
      />
    </div>

    <button type="submit" className={btnPrimary}>Regi
strarse</button>

    {errors.root && <p className={\`${errorClass} text
-center`} >{errors.root.message}</p>}
  </form>
</div>
</section>
);
};

export default Register;

```

8.2 Login

```
import { useForm } from "react-hook-form";
import { useNavigate } from "react-router-dom";
import { loginUser } from "../services/api";
import { useAuth } from "../context/AuthContext";

const inputClass = "w-full px-3 py-2 rounded-lg bg-input border border-text-white placeholder:text-muted focus:outline-none focus:border-brand transition-colors";
const errorClass = "text-error text-xs mt-1";
const btnPrimary = "w-full px-4 py-2.5 rounded-lg bg-brand hover:bg-brand-dark text-white uppercase tracking-widest text-sm font-medium transition-colors";

const Login = () => {
  const navigate = useNavigate();
  const { login } = useAuth();
  const { register, handleSubmit, formState: { errors }, setError } = useForm();

  const onSubmit = async (data) => {
    const res = await loginUser(data);

    if (res.error) {
      // el backend devuelve { error } para credenciales incorrectas
      setError("root", { message: res.error });
      return;
    }

    login(res.token, res.user);
    navigate("/dashboard");
  };

  return (
    <section className="flex flex-col items-center justify-center min-h-[80vh] px-4">
```

```

    <div className="w-full max-w-sm bg-surface rounded-2xl p-8 border border-border">
      <h1 className="text-brand text-2xl font-bold uppercase tracking-widest text-center mb-6">
        Login
      </h1>

      <form onSubmit={handleSubmit(onSubmit)} className="flex flex-col gap-4">
        <div>
          <input type="email" placeholder="Email" className={inputClass}
            {...register("email", { required: "El email es obligatorio" })}
          />
          {errors.email && <p className={errorClass}>{errors.email.message}</p>}
        </div>

        <div>
          <input type="password" placeholder="Contraseña" className={inputClass}
            {...register("password", { required: "La contraseña es obligatoria" })}
          />
          {errors.password && <p className={errorClass}>{errors.password.message}</p>}
        </div>

        <button type="submit" className={btnPrimary}>Entrar</button>

        {errors.root && <p className={` ${errorClass} text-center`}>{errors.root.message}</p>}
      </form>
    </div>
  </section>
);

```



```
};

export default Login;
```

8.3 Dashboard

```
import { useCallback, useEffect, useState } from "react";
import { useAuth } from "../context/AuthContext";
import { getTasks, createTask, updateTask, deleteTask } from "../services/api";
import TaskForm from "../components/TaskForm";
import TaskFilters from "../components/TaskFilters";
import TaskList from "../components/TaskList";

const Dashboard = () => {
  const { token, user } = useAuth();
  const [tasks, setTasks] = useState([]);
  const [taskToEdit, setTaskToEdit] = useState(null);
  const [filters, setFilters] = useState({ status: "", date: "", sort: "" });

  const loadTasks = useCallback(async () => {
    const res = await getTasks(token, filters);
    // el backend devuelve el array directamente; || [] protege si llega undefined
    setTasks(res || []);
  }, [token, filters]);

  useEffect(() => { loadTasks(); }, [loadTasks]);

  const handleSubmitTask = useCallback(async (formData) => {
    if (taskToEdit) {
      await updateTask(token, taskToEdit._id, formData);
      setTaskToEdit(null);
    } else {
      await createTask(token, formData);
    }
  });
```

```

    loadTasks();
  }, [token, taskToEdit, loadTasks]));

  const handleDeleteTask = useCallback(async (id) => {
    await deleteTask(token, id);
    loadTasks();
  }, [token, loadTasks]);

  const clearTaskToEdit = useCallback(() => setTaskToEdit(null), []);

  return (
    <section className="flex flex-col items-center gap-8 px-4 py-10 max-w-5xl mx-auto w-full">
      <div className="text-center">
        <h1 className="text-brand text-3xl font-bold uppercase tracking-widest">Dashboard</h1>
        <p className="text-muted mt-1">Hola, {user?.username}</p>
      </div>

      <TaskForm
        onSubmitTask={handleSubmitTask}
        taskToEdit={taskToEdit}
        clearTaskToEdit={clearTaskToEdit}
      />

      <TaskFilters filters={filters} setFilters={setFilters} />

      <TaskList tasks={tasks} onEdit={setTaskToEdit} onDelete={handleDeleteTask} />
    </section>
  );
};

export default Dashboard;

```

8.4 Formularios del perfil

UsernameForm

```
import { useEffect } from "react";
import { useForm } from "react-hook-form";
import { updateUsername } from "../services/api";
import { useAuth } from "../context/AuthContext";

const inputClass = "w-full px-3 py-2 rounded-lg bg-input border border-border text-white placeholder:text-muted focus:outline-none focus:border-brand transition-colors";
const errorClass = "text-error text-xs mt-1";
const btnClass = "w-full px-4 py-2 rounded-lg bg-brand hover:bg-brand-dark text-white uppercase tracking-widest text-sm font-medium transition-colors";

const UsernameForm = ({ onMessage }) => {
  const { token, user, loadUser } = useAuth();
  const { register, handleSubmit, formState: { errors }, setError, reset } = useForm({
    defaultValues: { username: user?.username || "" },
  });

  // si loadUser recarga el usuario, sincronizamos el input
  useEffect(() => { reset({ username: user?.username || "" }); }, [user, reset]);

  const onSubmit = async (data) => {
    const res = await updateUsername(token, data.username);
    if (res.error) { setError("root", { message: res.error }); return; }
    loadUser();
    onMessage("Username modificado correctamente");
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="flex
```

```

flex-col gap-3">
  <h2 className="text-brand font-semibold uppercase tra
cking-widest text-sm">Cambiar username</h2>
  <div>
    <input type="text" className={inputClass}
      {...register("username", {
        required: "El username es obligatorio",
        minLength: { value: 3, message: "Mínimo 3 carac
teres" },
      })}
    />
    {errors.username && <p className={errorClass}>{erro
rs.username.message}</p>}
    {errors.root && <p className={errorClass}>{erro
rs.root.message}</p>}
  </div>
  <button type="submit" className={btnClass}>Guardar us
ername</button>
</form>
);
};

export default UsernameForm;

```

PasswordForm

Gana un campo de **confirmación de contraseña** validado en el cliente con `watch` + `validate`. El backend no necesita cambios.

```

import { useForm } from "react-hook-form";
import { updatePassword } from "../services/api";
import { useAuth } from "../context/AuthContext";

const inputClass = "w-full px-3 py-2 rounded-lg bg-input bo
rder border-border text-white placeholder:text-muted focus:
outline-none focus:border-brand transition-colors";
const errorClass = "text-error text-xs mt-1";
const btnClass = "w-full px-4 py-2 rounded-lg bg-brand ho

```

```
ver:bg-brand-dark text-white uppercase tracking-widest text-sm font-medium transition-colors";
```

```
const PasswordForm = ({ onMessage }) => {  
  const { token } = useAuth();  
  const { register, handleSubmit, formState: { errors }, setError, reset, watch } = useForm();
```

```
  // watch nos permite leer el valor de newPassword para validar que la confirmación coincide
```

```
  const newPassword = watch("newPassword");
```

```
  const onSubmit = async (data) => {  
    const res = await updatePassword(token, data.currentPassword, data.newPassword);
```

```
    // el backend devuelve { error } tanto para campos vacíos como para contraseña incorrecta
```

```
    if (res.error) {  
      setError("root", { message: res.error });  
      return;  
    }
```

```
    onMessage(res.message);  
    reset();  
  };
```

```
  return (  
    <form onSubmit={handleSubmit(onSubmit)} className="flex flex-col gap-3">  
      <h2 className="text-brand font-semibold uppercase tracking-widest text-sm">Cambiar contraseña</h2>
```

```
      <div>  
        <input type="password" placeholder="Contraseña actual" className={inputClass}  
          {...register("currentPassword", { required: "Introduce tu contraseña actual" })}>
```

```

        />
        {errors.currentPassword && <p className={errorClass}>{errors.currentPassword.message}</p>}
    </div>

    <div>
        <input type="password" placeholder="Contraseña nueva" className={inputClass}
            {...register("newPassword", {
                required: "Introduce la nueva contraseña",
                minLength: { value: 8, message: "Mínimo 8 caracteres" },
            })}
        />
        {errors.newPassword && <p className={errorClass}>{errors.newPassword.message}</p>}
    </div>

    <div>
        <input type="password" placeholder="Confirmar contraseña nueva" className={inputClass}
            {...register("confirmPassword", {
                required: "Confirma la nueva contraseña",
                validate: (value) => value === newPassword || "Las contraseñas no coinciden",
            })}
        />
        {errors.confirmPassword && <p className={errorClass}>{errors.confirmPassword.message}</p>}
    </div>

    <button type="submit" className={btnClass}>Guardar contraseña</button>
    {errors.root && <p className={errorClass}>{errors.root.message}</p>}
</form>
);
};

```

```
export default PasswordForm;
```

AvatarForm

```
import { useForm } from "react-hook-form";
import { updateAvatar } from "../services/api";
import { useAuth } from "../context/AuthContext";

const inputClass = "w-full px-3 py-2 rounded-lg bg-input border border-border text-white placeholder:text-muted focus:outline-none focus:border-brand transition-colors file:mr-3 file:text-muted file:bg-transparent file:border-0";
const errorClass = "text-error text-xs mt-1";
const btnClass = "w-full px-4 py-2 rounded-lg bg-brand hover:bg-brand-dark text-white uppercase tracking-widest text-sm font-medium transition-colors";

const AvatarForm = ({ onMessage }) => {
  const { token, loadUser } = useAuth();
  const { register, handleSubmit, formState: { errors }, setError, reset } = useForm();

  const onSubmit = async (data) => {
    if (!data.avatar?.[0]) {
      setError("avatar", { message: "Selecciona una imagen" });
    }
    return;

    const formData = new FormData();
    formData.append("avatar", data.avatar[0]);

    const res = await updateAvatar(token, formData);
    if (res.error) { setError("root", { message: res.error }); }
    return;
  }
}
```

```

    // recargamos el usuario para que el <img> del perfil r
    efleje el nuevo avatar
    loadUser();
    onMessage("Avatar actualizado correctamente");
    reset();
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)} className="flex
flex-col gap-3">
      <h2 className="text-brand font-semibold uppercase tra
cking-widest text-sm">Cambiar avatar</h2>
      <div>
        <input type="file" accept="image/*" className={inpu
tClass} {...register("avatar")} />
        {errors.avatar && <p className={errorClass}>{error
s.avatar.message}</p>}
        {errors.root && <p className={errorClass}>{error
s.root.message}</p>}
      </div>
      <button type="submit" className={btnClass}>Guardar av
atar</button>
    </form>
  );
};

export default AvatarForm;

```

8.5 Profile

```

import { useState } from "react";
import { useAuth } from "../context/AuthContext";
import UsernameForm from "../components/UsernameForm";
import PasswordForm from "../components/PasswordForm";
import AvatarForm from "../components/AvatarForm";

const Profile = () => {
  const { user } = useAuth();

```



```

const [message, setMessage] = useState("");

return (
  <section className="flex flex-col items-center gap-8 px-4 py-10 max-w-lg mx-auto w-full">
    <h1 className="text-brand text-3xl font-bold uppercase tracking-widest">Perfil</h1>

    {user?.avatar && (
      <img
        src={user.avatar}
        alt={user.username}
        className="w-28 h-28 rounded-full object-cover border-4 border-brand"
      />
    )}

    {message && (
      <p className="text-success text-sm bg-green-900/20 border border-green-900 rounded-lg px-4 py-2 w-full text-center">
        {message}
      </p>
    )}

    <div className="w-full flex flex-col gap-6 bg-surface rounded-2xl p-6 border border-brand">
      <UsernameForm onMessage={setMessage} />
      <hr className="border-brand" />
      <PasswordForm onMessage={setMessage} />
      <hr className="border-brand" />
      <AvatarForm onMessage={setMessage} />
    </div>
  </section>
);
};

export default Profile;

```

9. Layout y rutas

9.1 App.jsx

```
import { Outlet, useNavigate } from "react-router-dom";
import Navbar from "../components/Navbar";
import { useAuth } from "../context/AuthContext";

const App = () => {
  const { token, logout } = useAuth();
  const navigate = useNavigate();

  const handleLogout = () => {
    logout();
    navigate("/login");
  };

  return (
    <div className="min-h-screen bg-bg text-white">
      <Navbar token={token} onLogout={handleLogout} />
      <main>
        <Outlet />
      </main>
    </div>
  );
};

export default App;
```

9.2 main.jsx

```
import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import "../index.css";
import App from "../App.jsx";

import { AuthProvider } from "../context/AuthContext.jsx";
```

```

import ProtectedRoute    from "../components/ProtectedRoute.
jsx";
import Home              from "../pages/Home.jsx";
import Login             from "../pages/Login.jsx";
import Register          from "../pages/Register.jsx";
import Dashboard         from "../pages/Dashboard.jsx";
import Profile           from "../pages/Profile.jsx";
import NotFound          from "../pages/NotFound.jsx";
import { BrowserRouter, Routes, Route } from "react-router-
dom";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <BrowserRouter>
      <AuthProvider>
        <Routes>
          <Route path="/" element={<App />}>
            <Route index          element={<Home />} />
            <Route path="/login"   element={<Login />} />
            <Route path="/register" element={<Register />}
          />

          <Route element={<ProtectedRoute />}>
            <Route path="/dashboard" element={<Dashboard
          />} />

            <Route path="/profile"  element={<Profile /
          >} />

          </Route>

          <Route path="*" element={<NotFound />} />
        </Route>
      </Routes>
    </AuthProvider>
  </BrowserRouter>
</StrictMode>
);

```

10. Cómo arrancar el frontend

```
npm run dev
```

Vite levantará el servidor en `http://localhost:5173` y se comunicará con el backend en `http://localhost:3000/api` (leído desde `.env`).

11. Flujo completo de la aplicación

1. el usuario entra en la portada (Home)
2. se registra — RHF valida en cliente; si no hay avatar, el backend pone el placeholder
3. inicia sesión — el token se guarda en `localStorage`
4. `loadUser` carga el perfil desde el backend
5. entra al dashboard — puede crear, editar, borrar, filtrar y ordenar tareas
6. en el perfil puede cambiar nombre, contraseña (con confirmación) y avatar
7. al cambiar el avatar, el backend borra el antiguo de Cloudinary automáticamente
8. `loadUser` se llama tras cambiar nombre o avatar — la UI refleja el cambio sin recargar

12. ESLint

```
npm install -D eslint-plugin-react
```

`eslint.config.js`:

```
import globals from "globals";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
import react from "eslint-plugin-react";

export default [
  { ignores: ["dist"] },
  {
    rules: {
      "react-refresh/only-export-components": [
        "warn",
        { allowConstantExport: true },
      ],
    },
  },
  reactHooks,
  reactRefresh,
  react,
];
```

```

    files: ["**/*.{js,jsx}"],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true }, sourceType: "module" },
    },
    plugins: { "react-hooks": reactHooks, "react-refresh": reactRefresh, react },
    rules: {
      ...reactHooks.configs.recommended.rules,
      "react-refresh/only-export-components": ["warn", { allowConstantExport: true }],
      "react/prop-types": "off",
      "react/jsx-key": "error",
      "react/self-closing-comp": "warn",
      "react/jsx-no-duplicate-props": "error",
      "no-console": "warn",
      "no-unused-vars": ["warn", { argsIgnorePattern: "^_" }],
      "eqeqeq": ["error", "always"],
      "prefer-const": "error",
      "no-var": "error",
    },
  },
];

```

`package.json` scripts:

```

"scripts": {
  "dev": "vite",
  "build": "vite build",
  "lint": "eslint .",
  "lint:fix": "eslint . --fix",
  "test": "vitest",
  "test:ui": "vitest --ui",
  "preview": "vite preview"
}

```

13. Tests — Vitest + Testing Library

```
npm install -D vitest @vitest/ui @testing-library/react @testing-library/jest-dom @testing-library/user-event jsdom
```

`@vitest/ui` es necesario para el script `test:ui`. Sin él, `npm run test:ui` falla.

`src/test/setup.js`:

```
import "@testing-library/jest-dom";
```

Test de TaskCard

```
import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import TaskCard from "../components/TaskCard";

const mockTask = { _id: "abc123", title: "Estudiar React",
date: "2026-04-14T00:00:00.000Z", status: "pending" };

describe("TaskCard", () => {
  it("muestra título, fecha y estado", () => {
    render(<TaskCard task={mockTask} onEdit={() => {}} onDelete={() => {}} />);
    expect(screen.getByText("Estudiar React")).toBeInTheDocument();
    expect(screen.getByText("2026-04-14")).toBeInTheDocument();
    expect(screen.getByText("Pendiente")).toBeInTheDocument();
  });

  it("llama a onEdit con la tarea al hacer click en Editar", async () => {
    const handleEdit = vi.fn();
    render(<TaskCard task={mockTask} onEdit={handleEdit} onDelete={() => {}} />);
```

```

    await userEvent.click(screen.getByRole("button", { name: /editar/i }));
    expect(handleEdit).toHaveBeenCalledWith(mockTask);
  });

  it("llama a onDelete con el _id al hacer click en Borrar", async () => {
    const handleDelete = vi.fn();
    render(<TaskCard task={mockTask} onEdit={() => {}} onDelete={handleDelete} />);
    await userEvent.click(screen.getByRole("button", { name: /borrar/i }));
    expect(handleDelete).toHaveBeenCalledWith("abc123");
  });
});

```

Test de TaskList

```

import { render, screen } from "@testing-library/react";
import TaskList from "../components/TaskList";

const mockTasks = [
  { _id: "1", title: "Tarea uno", date: "2026-04-10T00:00:00.000Z", status: "pending" },
  { _id: "2", title: "Tarea dos", date: "2026-04-11T00:00:00.000Z", status: "completed" },
];

describe("TaskList", () => {
  it("muestra el mensaje vacío cuando no hay tareas", () => {
    render(<TaskList tasks={[]} onEdit={() => {}} onDelete={() => {}} />);
    expect(screen.getByText(/no hay tareas/i)).toBeInTheDocument();
  });

  it("renderiza una carta por cada tarea", () => {

```

```

    render(<TaskList tasks={mockTasks} onEdit={() => {}} on
Delete={() => {}} />);
    expect(screen.getByText("Tarea uno")).toBeInTheDocument
    ();
    expect(screen.getByText("Tarea dos")).toBeInTheDocument
    ();
    });

    it("renderiza tantos botones Editar como tareas", () => {
    render(<TaskList tasks={mockTasks} onEdit={() => {}} on
Delete={() => {}} />);
    expect(screen.getAllByRole("button", { name: /editar/i
})).toHaveLength(2);
    });
    });

```

Test de TaskForm

```

import { render, screen, waitFor } from "@testing-library/r
eact";
import userEvent from "@testing-library/user-event";
import TaskForm from "../components/TaskForm";

describe("TaskForm", () => {
    it("muestra 'Crear tarea' cuando no hay tarea a editar",
    () => {
        render(<TaskForm onSubmitTask={() => {}} taskToEdit={nu
ll} clearTaskToEdit={() => {}} />);
        expect(screen.getByRole("button", { name: /crear tarea/
i })).toBeInTheDocument();
        expect(screen.queryByRole("button", { name: /cancelar/i
})).not.toBeInTheDocument();
    });

    it("muestra 'Guardar cambios' y cancelar cuando hay tarea
a editar", () => {
        const task = { _id: "x", title: "Test", date: "2026-04-
14T00:00:00.000Z", status: "pending" };

```



```

    render(<TaskForm onSubmitTask={() => {}} taskToEdit={task}
clearTaskToEdit={() => {}} />);
    expect(screen.getByRole("button", { name: /guardar cambios/i })).toBeInTheDocument();
    expect(screen.getByRole("button", { name: /cancelar/i })).toBeInTheDocument();
  });

  it("llama a onSubmitTask con los datos al enviar", async
() => {
    const handleSubmit = vi.fn();
    render(<TaskForm onSubmitTask={handleSubmit} taskToEdit=
{null} clearTaskToEdit={() => {}} />);

    await userEvent.type(screen.getByPlaceholderText(/título/i), "Mi tarea");
    const dateInput = document.querySelector("input[type='date']");
    await userEvent.type(dateInput, "2026-04-14");
    await userEvent.click(screen.getByRole("button", { name: /crear tarea/i }));

    // waitFor porque handleSubmit de RHF ejecuta la validación de forma asíncrona
    await waitFor(() => {
      expect(handleSubmit).toHaveBeenCalledWith(expect.objectContaining({ title: "Mi tarea" }));
    });
  });

  it("muestra error de validación si el título está vacío",
async () => {
    render(<TaskForm onSubmitTask={() => {}} taskToEdit={null}
clearTaskToEdit={() => {}} />);
    await userEvent.click(screen.getByRole("button", { name: /crear tarea/i }));
    await waitFor(() => {
      expect(screen.getByText(/el título es obligatorio/

```

```

i)).toBeInTheDocument();
    });
  });

  it("llama a clearTaskToEdit al pulsar cancelar", async ()
=> {
    const handleClear = vi.fn();
    const task = { _id: "x", title: "Test", date: "2026-04-14T00:00:00.000Z", status: "pending" };
    render(<TaskForm onSubmitTask={() => {}} taskToEdit={task} clearTaskToEdit={handleClear} />);
    await userEvent.click(screen.getByRole("button", { name: /cancelar/i }));
    expect(handleClear).toHaveBeenCalledTimes(1);
  });
});

```

Test de TaskFilters

```

import { render, screen } from "@testing-library/react";
import userEvent from "@testing-library/user-event";
import TaskFilters from "../components/TaskFilters";

const defaultFilters = { status: "", date: "", sort: "" };

describe("TaskFilters", () => {
  it("renderiza dos selectores", () => {
    render(<TaskFilters filters={defaultFilters} setFilters={() => {}} />);
    expect(screen.getAllByRole("combobox")).toHaveLength(2);
  });

  it("llama a setFilters al cambiar el estado", async () => {
    const setFilters = vi.fn();
    render(<TaskFilters filters={defaultFilters} setFilters={setFilters} />);
  });
});

```

```
const [statusSelect] = screen.getAllByRole("combobox");
await userEvent.selectOptions(statusSelect, "pending");
expect(setFilters).toHaveBeenCalledWith({ ...defaultFilters, status: "pending" });
});
});
```

```
npm run test      # modo watch
npm run test:ui   # interfaz visual
```

14. Despliegue — Vercel (frontend) y Render (backend)

14.1 Backend en Render

Sube el backend a GitHub. En render.com, crea un **Web Service**:

```
Branch:      main
Runtime:     Node
Build Command: npm install
Start Command: node index.js
Plan:       Free
```

Variables de entorno en Render:

```
DB_URL           → tu URL de MongoDB Atlas
JWT_SECRET       → clave secreta larga y aleatoria
CLOUDINARY_CLOUD_NAME → tu cloud name
CLOUDINARY_API_KEY   → tu API key
CLOUDINARY_API_SECRET → tu API secret
FRONTEND_URL       → https://tu-proyecto.vercel.app
```

❗ No añadas `PORT` — Render lo gestiona internamente.

El CORS del backend ya usa `process.env.FRONTEND_URL`, así que con solo añadir la variable en Render queda configurado sin tocar el código.

14.2 Frontend en Vercel

Crea el archivo `.env.production` en la raíz del frontend (no lo subas a Git):

```
VITE_API_URL=https://doitapp-backend.onrender.com/api
```

Haz push a GitHub. En vercel.com importa el proyecto y añade la variable de entorno:

```
VITE_API_URL → https://doitapp-backend.onrender.com/api
```

Crea `vercel.json` en la raíz del frontend para que React Router funcione al recargar:

```
{
  "rewrites": [
    { "source": "/(.*)", "destination": "/index.html" }
  ]
}
```

Sin esto, recargar `/dashboard` devuelve un 404 de Vercel porque no existe el archivo `dashboard.html`.

14.3 Tabla de URLs y variables

```
Backend local   → http://localhost:3000
Backend Render  → https://doitapp-backend.onrender.com
Frontend local  → http://localhost:5173
Frontend Vercel → https://doitapp-frontend.vercel.app
```

Variable	Dónde
DB_URL	Render → Environment
JWT_SECRET	Render → Environment
CLOUDINARY_*	Render → Environment
FRONTEND_URL	Render → Environment
VITE_API_URL	Vercel → Environment Variables

14.4 Checklist de despliegue

- Backend en Render con todas las variables de entorno
- FRONTEND_URL apunta a la URL real de Vercel
- VITE_API_URL apunta a la URL real de Render
- vercel.json está en el repo del frontend
- npm run build no da errores
- Registro y login funcionan en producción
- Subida de avatar y limpieza de Cloudinary funcionan
- Dashboard carga las tareas correctamente
- Recargar /dashboard no da 404

Resumen técnico

Backend

- Node.js + Express con estructura modular: `config`, `middlewares`, `models`, `controllers`, `routes`, `utils`
- MongoDB + Mongoose con dos colecciones: `users` y `tasks`
- JWT para autenticación, bcrypt para contraseñas
- Cloudinary para avatares: sube a `doitusers`, elimina el anterior al actualizar
- Manejo centralizado de errores: errores esperados responden directamente con `error` o `message`; errores inesperados van a `utils/errorHandler` vía `next(error)`
- CORS configurable vía `FRONTEND_URL` en el `.env`

Frontend

- React 19 + Vite con nombre de proyecto `doitapp`
- React Router 7: rutas públicas y privadas con `ProtectedRoute` + `Outlet`
- react-hook-form en todos los formularios: sin `useState` por campo, validación declarativa en `register()`, errores del backend en `setError("root")`
- Tailwind CSS v4 con `@tailwindcss/vite`: tema custom definido en `@theme` con colores `brand`, `surface`, `input`, `muted`, `error`
- `AuthContext` expone `loadUser` para recargar el usuario sin recargar la página
- URL del backend en `import.meta.env.VITE_API_URL` (`.env` en local, Vercel en producción)
- Memoización con `memo` y `useCallback` en componentes y handlers

- Tests con Vitest + Testing Library para `TaskCard` , `TaskList` , `TaskForm` y `TaskFilters`

Despliegue

- Backend en Render (Node, Free plan)
- Frontend en Vercel con `vercel.json` para que React Router funcione al recargar